

Course : Machine Learning
Lecture On : ANN
Instructor : Shambhu Kumar

KEY TOPICS

- ❖ Perceptron
- ❖ Introduction to ANN
- ❖ Feedforward
- ❖ Back-Propagation
- ❖ Batch and Batch Training
- ❖ Open Q&A

- One of the earliest proposed (1960's) simple device towards building a ANN. It uses Step function as Activation function.
- The perceptron takes a **weighted sum** of multiple inputs (along with a bias) as the cumulative input and applies a **step function** on the cumulative input, i.e. it returns 1 if the input is positive, else -1.

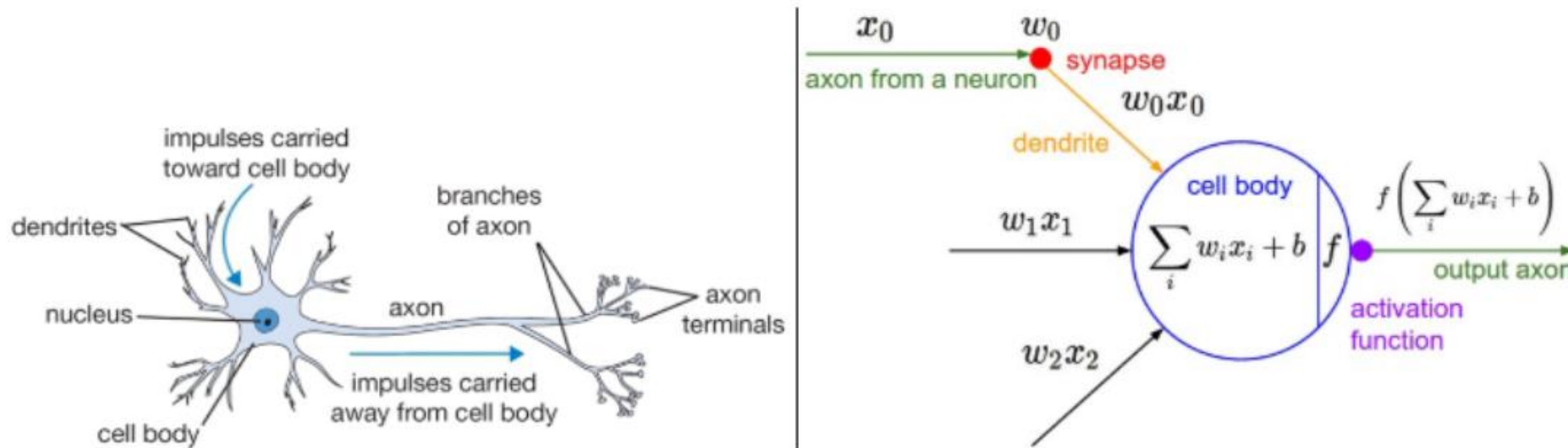
$$\text{Cumulative Input} = w^T \cdot x + b = w_1x_1 + w_2x_2 + \dots + w_kx_k + b$$

- Due to Step functions used, It works in binary mode – either fires or doesn't fire (based on hard-cut threshold)
- Originally, A single Perceptron were designed to be a classifier.

If $y(w^T \cdot x + b) > 0$: **Class I** else **Class II**

- However, a network of perceptron can be used to achieve much more complex task (**universal function estimator**) e.g. multi-class classification. The stacked perceptron architecture is also known as **MLP**.

- An ANN is a collection of many simple deices called Neurons. An artificial **neuron** is very similar to a **perceptron**, except that the activation function is not a step function.
- An ANN (or NN) simply try to mimic the working or functioning of human brains.
- The data in the network flows through each neuron by a connection. Every connection has a specific weight by which the flow of data is regulated.



A cartoon drawing of a biological neuron (left) and its mathematical model (right).

Why we use Activation functions

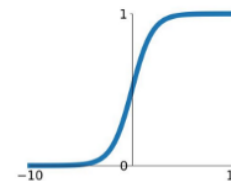
Our brain functions non-linearly for complex calculation, high parallelism, and for ability to handle imprecise and fuzzy information.

Activation functions are really important for a Artificial Neural Network to learn and make sense Non-linear complex functional mappings between the inputs and response variable.

AF introduce non-linear properties to our Network. Their **main purpose is to convert a input signal of a node in a A-NN to an output signal**. That output signal now is used as a input in the next layer in the stack.

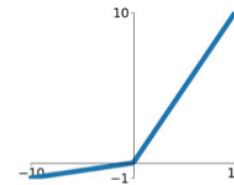
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



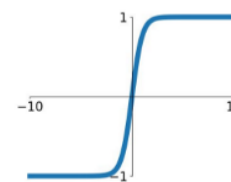
Leaky ReLU

$$\max(0.1x, x)$$



tanh

$$\tanh(x)$$

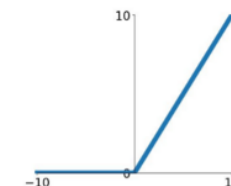


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

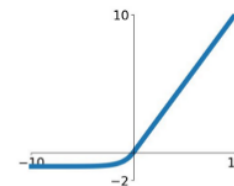
ReLU

$$\max(0, x)$$



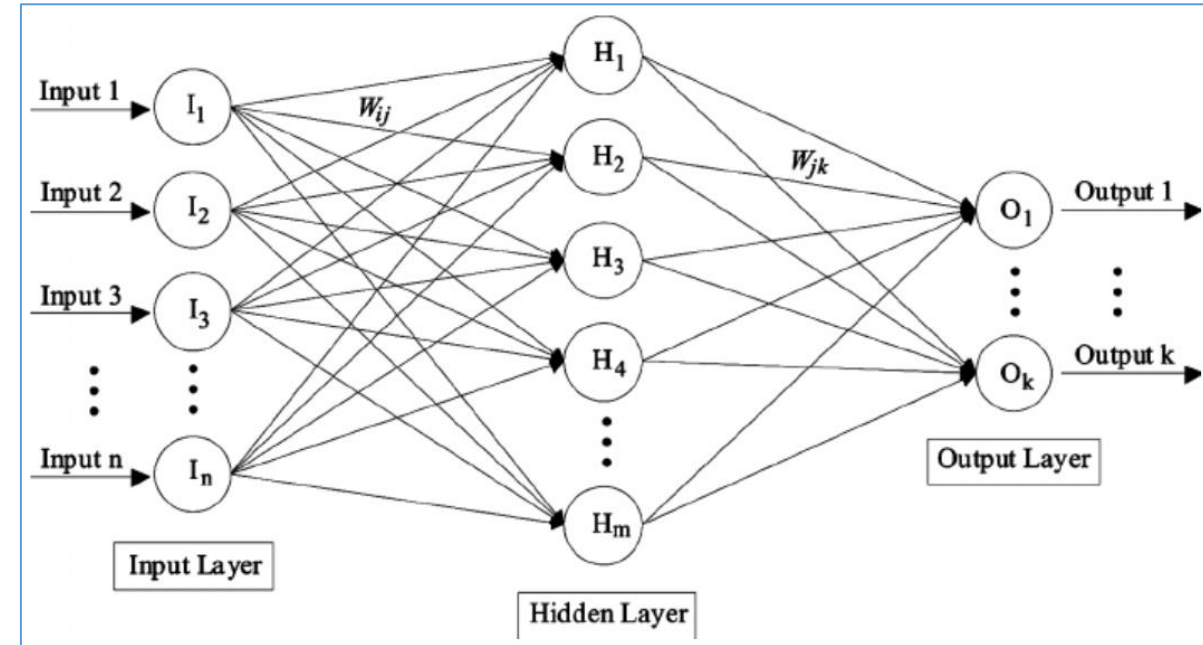
ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



<https://ai.stackexchange.com/questions/5493/what-is-the-purpose-of-an-activation-function-in-neural-networks>

1. Neurons are arranged in layers and the layers are arranged sequentially.
2. Neurons within the same layer do not interact with each other.
3. All the inputs enter the network through the input layer and all the outputs go out of the network through the output layer.
4. All neurons in layer L are connected to all neurons in layer $L+1$.
5. Every interconnection in the neural network has a weight and a bias associated with it.
6. All neurons in a particular layer use the same activation function.



Notes:

- i. ANNs only take **numerical inputs**.
- ii. ANNs – are trained on **weights and biases**
- iii. **Hyperparameters** : No. of Layers, No. of neurons in Input/Hidden/Output layers, activation function

Q1. Assume a simple MLP model with 3 neurons and inputs= 1,2,3. The weights to the input neurons are 4,5 and 6 respectively. Assume the activation function is a linear constant value of 3. What will be the output ?

A: 96 ($3(1*4+2*5+6*3)$)

Q3. If you increase the number of hidden layers in a Multi Layer Perceptron, the classification error of test data always decreases. True or False?

- A. True
- B. False

B: False (due to Overfitting)

Q2. For which values of w_1 , w_2 and b does a simple perceptron (step function) implement an AND function?

- A. Bias = -1.5, $w_1 = 1$, $w_2 = 1$
- B. Bias = 1.5, $w_1 = 2$, $w_2 = 2$
- C. Bias = 1, $w_1 = 1.5$, $w_2 = 1.5$
- D. None of these

Option A:

$$f(-1.5*1 + 1*0 + 1*0) = f(-1.5) = 0$$

$$f(-1.5*1 + 1*0 + 1*1) = f(-0.5) = 0$$

$$f(-1.5*1 + 1*1 + 1*0) = f(-0.5) = 0$$

$$f(-1.5*1 + 1*1 + 1*1) = f(0.5) = 1$$

- In a **feedforward network**, information always moves one direction; it never goes backwards or make a loop.
- In this case, we achieved output what we expected from the input without turning back or fine-tuning.
- FFN Algorithm:

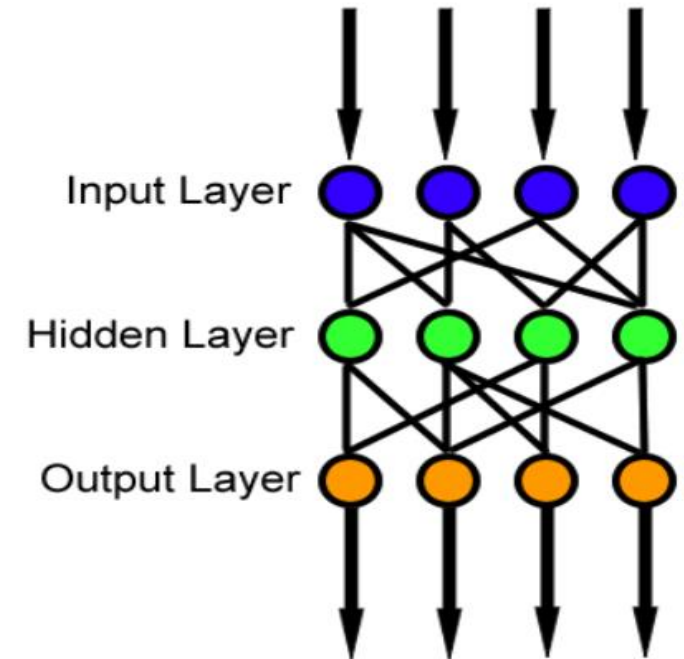
$$1. h^0 = x_i$$

2. for l in $[1, 2, \dots, L]$:

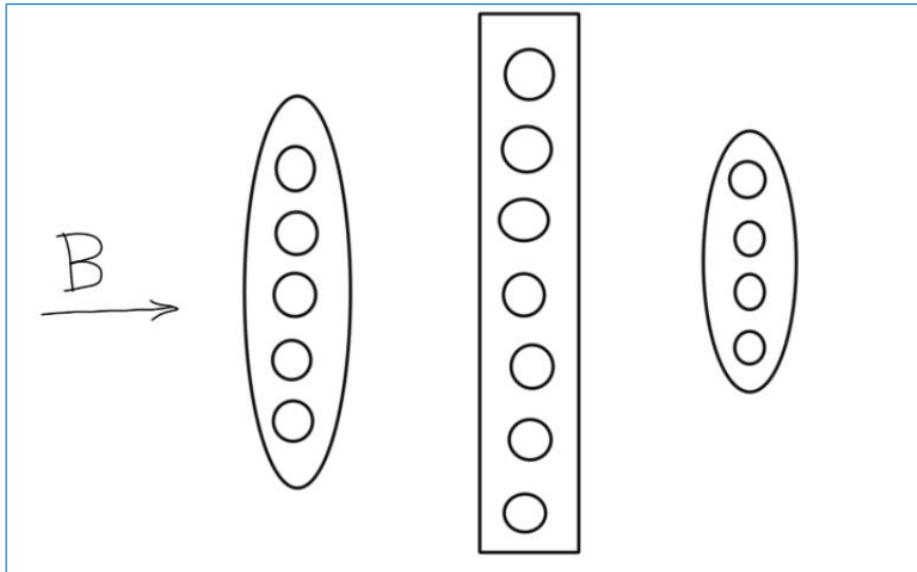
$$1. h^l = \sigma(W^l \cdot h^{l-1} + b^l)$$

$$3. p_i = e^{W^o \cdot h^L}$$

$$4. p_i = \text{normalize}(p_i)$$



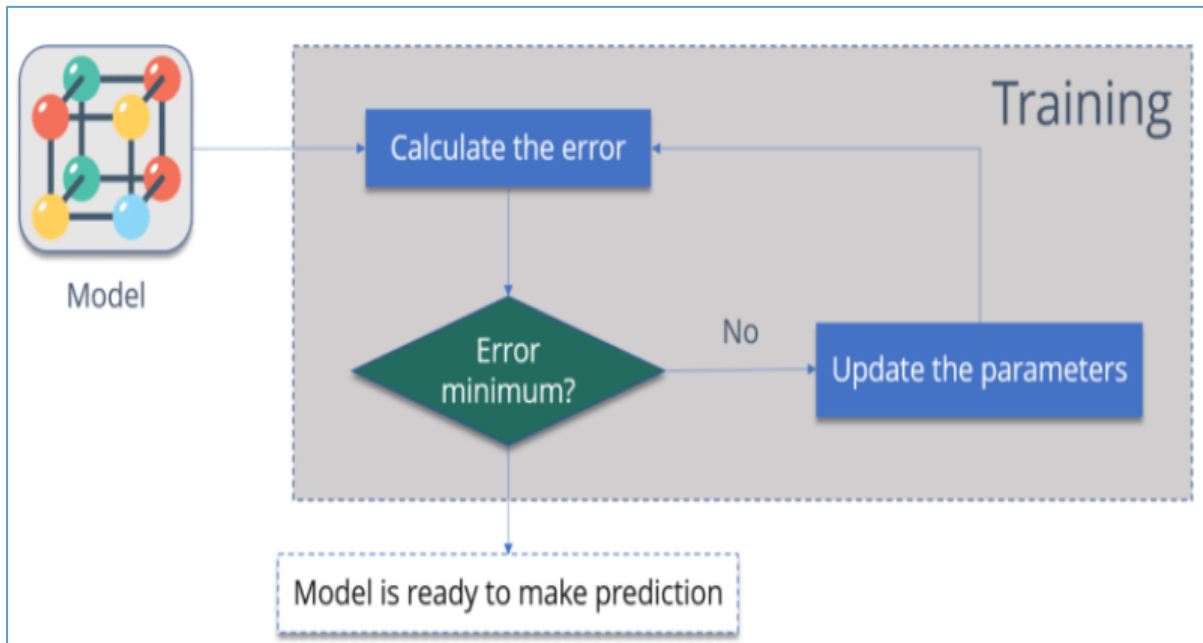
- Now, the entire calculation should happen for all the data points. This is almost impossible and computationally very expensive. Hence, in practice, the training happens in batches.
- A batch of size B is made of B data points and which is stacked side by side in a matrix. Thus, if the data points has ' d ' features, then size of batch B is (d, B) . This matrix is fed into ANN as input, also known as **Vectorized implementation**.



If $B = 32$, then

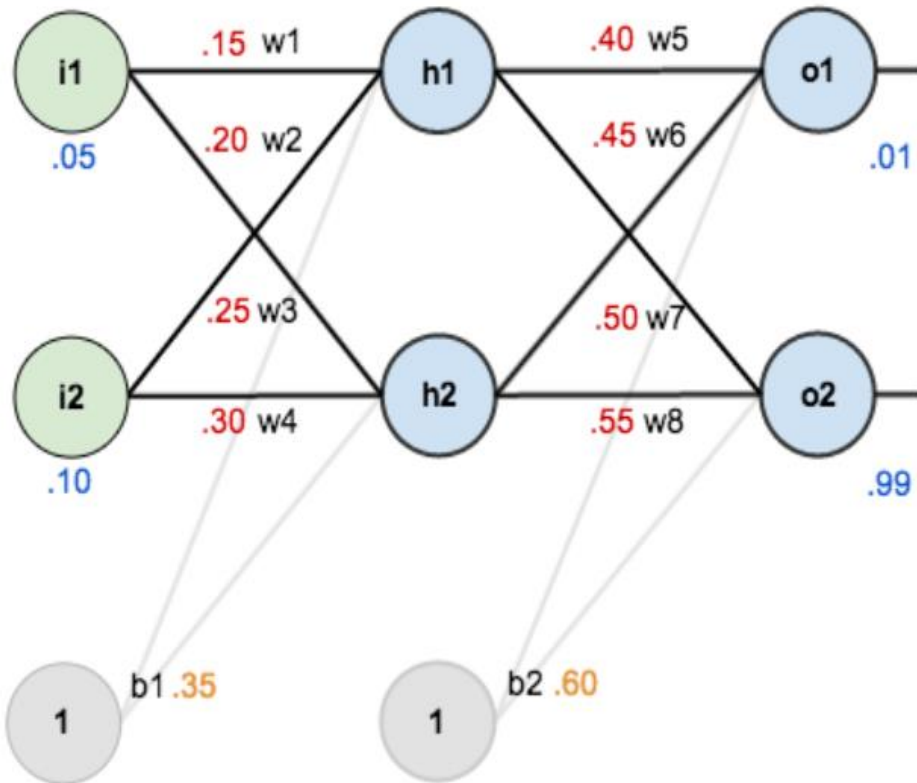
1. Size of B is: 1. 5×32
2. Dimension of $H1$ is: 2. 7×32
3. Dimension of P is: 3. 4×32

- Our first choice Weights and Bias might not be a good choice, so in order to minimize the average loss, we need to fine tune the W and B matrix.
- The objective behind training a NN is to minimize the overall cost - The desired output (output from the last layer) minus the actual output is the cost (or the loss), and we need to tune the parameters **W and B** such that the total cost is minimized.



- we can't directly calculate the derivative of the loss function with respect to the weights and biases because the equation of the loss function does not contain the weights and biases. Therefore, we need the **chain rule** to help us calculate it.

$$W_{new} = W_{old} - \alpha \cdot \frac{\partial L}{\partial W}$$



Step – 1: Forward Propagation:

Net Input For h1:

$$\text{net h1} = w1*i1 + w2*i2 + b1*1$$

$$\text{net h1} = 0.15*0.05 + 0.2*0.1 + 0.35*1 = 0.3775$$

Output Of h1:

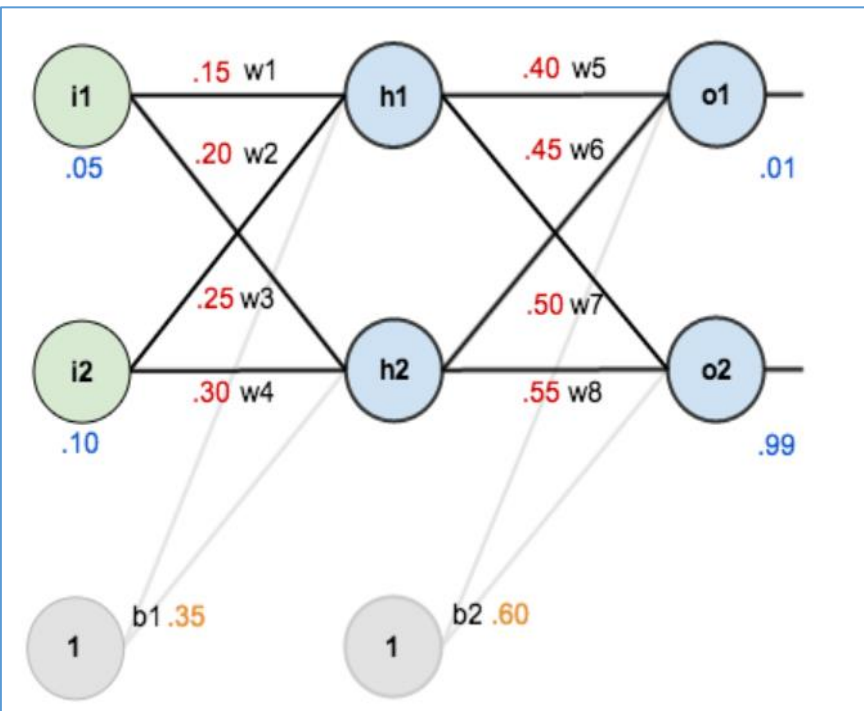
$$\text{out h1} = 1/1+e^{-\text{net h1}}$$

$$1/1+e^{-.3775} = 0.593269992$$

Output Of h2:

$$\text{out h2} = 0.596884378$$

- ✓ 2 input data point, 1 hidden layer with 2 neuron, and 2 neuron in the output layer
- ✓ There are total 8 weights in the N/W
- ✓ The bias $b1 = 0.35$, $b2 = 0.60$
- ✓ Input 1 = 0.05, Input 2 = 0.10
- ✓ Actual Output (Ground Truth) = **0.01 and 0.99**



Step – 1: Forward Propagation:

We will repeat this process for the output layer neurons, using the output from the hidden layer neurons as inputs.

We have already got:

Out h1 = 0.593269992

Out h2 = 0.596884378

Output For o1:

$$\text{net } o1 = w5 * \text{out } h1 + w6 * \text{out } h2 + b2 * 1$$

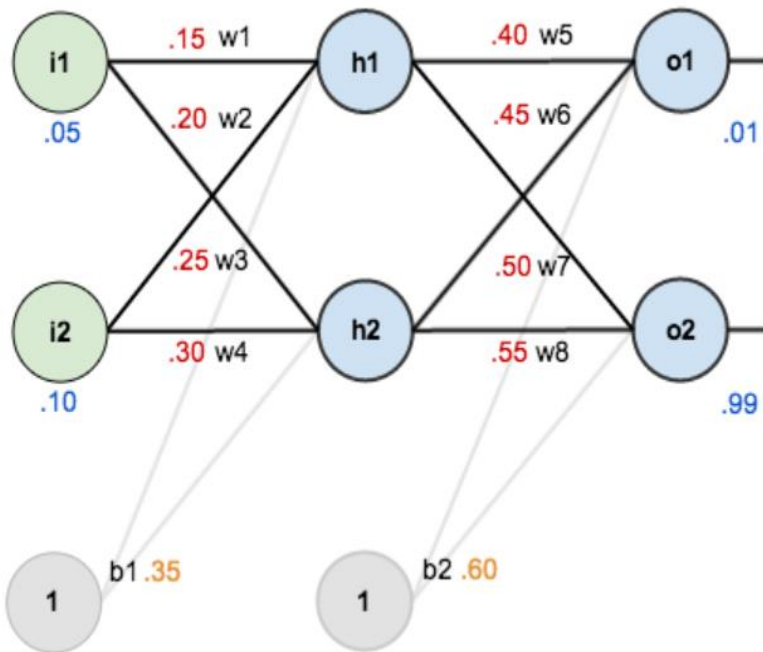
$$0.4 * 0.593269992 + 0.45 * 0.596884378 + 0.6 * 1 = 1.105905967$$

$$\text{Out } o1 = 1 / (1 + e^{-\text{net } o1})$$

$$1 / (1 + e^{-1.105905967}) = 0.75136507$$

Output For o2:

$$\text{Out } o2 = 0.772928465$$



Step –2: Calculating the Total Error

As we have calculated the final output (i.e. the predicted value), we can now calculate the total error. Let's say we use MSE as our loss function.

Error For o1:

$$E_{o1} = \sum 1/2(target - output)^2$$

$$\frac{1}{2} (0.01 - 0.75136507)^2 = 0.274811083$$

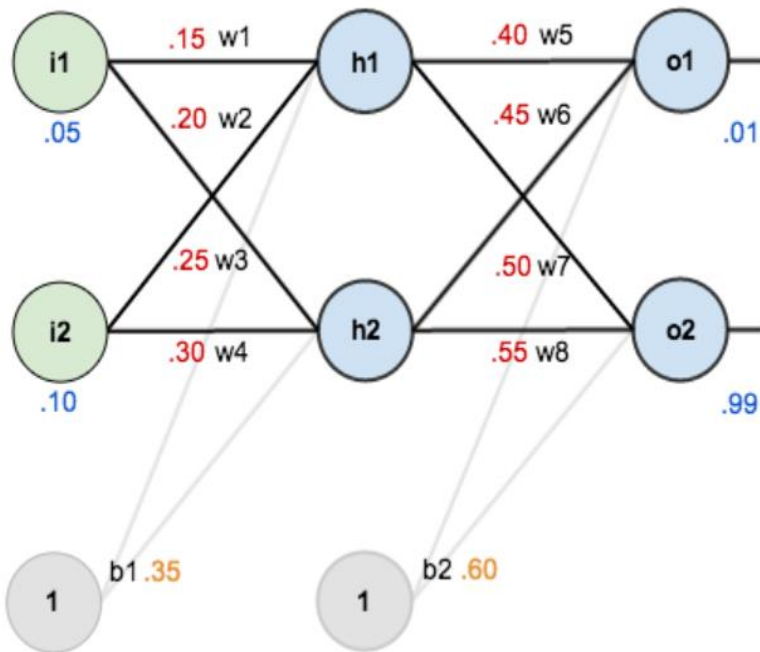
Error For o2:

$$E_{o2} = 0.023560026$$

Total Error:

$$E_{total} = E_{o1} + E_{o2}$$

$$0.274811083 + 0.023560026 = 0.298371109$$

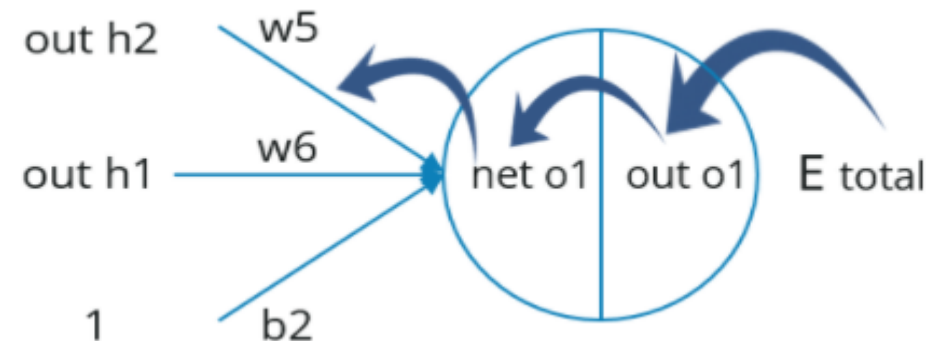


Step – 3: Backward Propagation

As we have calculated the total error now, we need to update each of the weights in the network so that they cause the actual output to be closer the target output, thereby minimizing the error for each output neuron and the network as a whole.

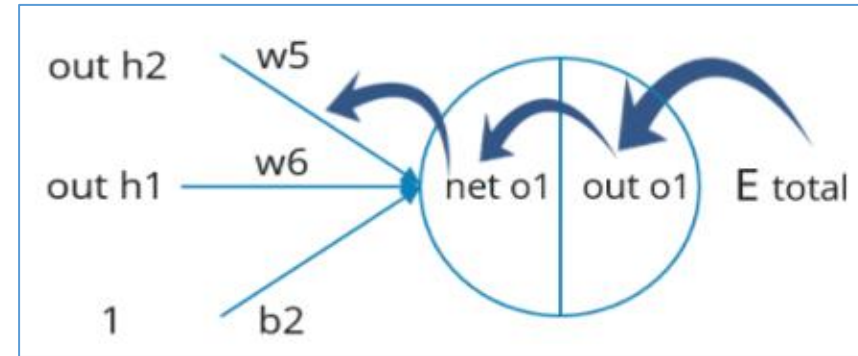
To start with, we can take **W5 as the starting point** and see how to calculate the updated W5. Similarly, we need to calculate W6, W7, W8 and then W1...W4. Now we need to find the contribution of W5 in the total error i.e. we want to know $\frac{\delta E_{total}}{\delta w_5}$

$$\frac{\delta E_{total}}{\delta w_5} = \frac{\delta E_{total}}{\delta out\ o1} * \frac{\delta out\ o1}{\delta net\ o1} * \frac{\delta net\ o1}{\delta w_5}$$



Step – 3: Backward Propagation

So, we need to calculate 3 different derivatives to know the impact of W5 on Total error:



$$E_{\text{total}} = 1/2(\text{target o1} - \text{out o1})^2 + 1/2(\text{target o2} - \text{out o2})^2$$

$$\frac{\delta E_{\text{total}}}{\delta \text{out o1}} = -(\text{target o1} - \text{out o1}) = -(0.01 - 0.75136507) = 0.74136507$$

$$\text{out o1} = 1 / (1 + e^{-\text{net o1}})$$

$$\frac{\delta \text{out o1}}{\delta \text{net o1}} = \text{out o1} (1 - \text{out o1}) = 0.75136507 (1 - 0.75136507) = 0.186815602$$

$$\text{net o1} = w5 * \text{out h1} + w6 * \text{out h2} + b2 * 1$$

$$\frac{\delta \text{net o1}}{\delta w5} = 1 * \text{out h1} * w5^{(1-1)} + 0 + 0 = 0.593269992$$

Step – 3: Backward Propagation

Now we have calculated all the derivatives, we can now calculate the rate of change of error w.r.t change in weight W5.

$$\frac{\delta E_{total}}{\delta w_5} = \frac{\delta E_{total}}{\delta out\ o1} * \frac{\delta out\ o1}{\delta net\ o1} * \frac{\delta net\ o1}{\delta w_5}$$

0.082167041

And then updated weight of W5 can be calculated as :

$$w_5^+ = w_5 - \eta \frac{\delta E_{total}}{\delta w_5}$$

$$w_5^+ = 0.4 - 0.5 * 0.082167041$$

Updated w5

0.35891648

Step – 3: Backward Propagation

Similarly, we can calculate $W6^*$, $W7^*$, and $W8^*$.

$$w_6^+ = 0.408666186$$

$$w_7^+ = 0.511301270$$

$$w_8^+ = 0.561370121$$

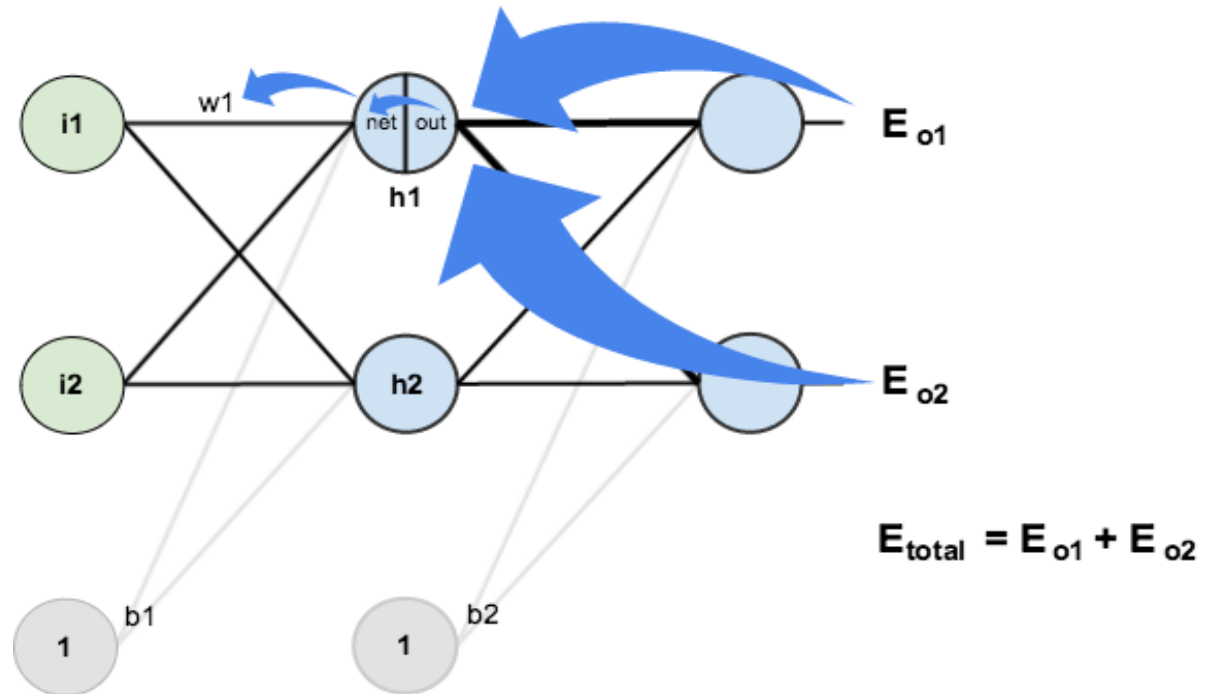
Step – 3: Backward Propagation to hidden Layer

Now, let's see how we can calculate the updated weight $W1$.

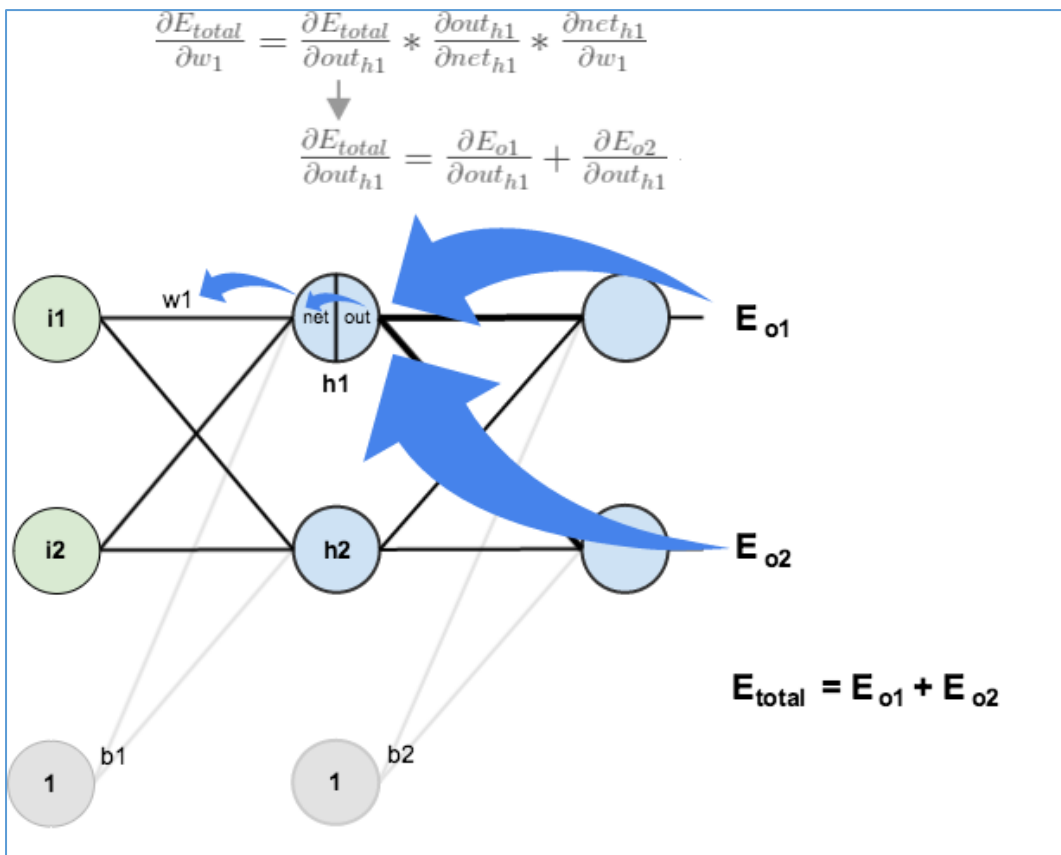
$$\frac{\partial E_{total}}{\partial w_1} = \frac{\partial E_{total}}{\partial out_{h1}} * \frac{\partial out_{h1}}{\partial net_{h1}} * \frac{\partial net_{h1}}{\partial w_1}$$

$$\downarrow$$

$$\frac{\partial E_{total}}{\partial out_{h1}} = \frac{\partial E_{o1}}{\partial out_{h1}} + \frac{\partial E_{o2}}{\partial out_{h1}}$$



Step – 3: Backward Propagation to one hidden Layer



Calculation of $\frac{\partial E_{o1}}{\partial out_{h1}}$

$$1. \quad \frac{\partial E_{o1}}{\partial out_{h1}} = \frac{\partial E_{o1}}{\partial net_{o1}} * \frac{\partial net_{o1}}{\partial out_{h1}}$$

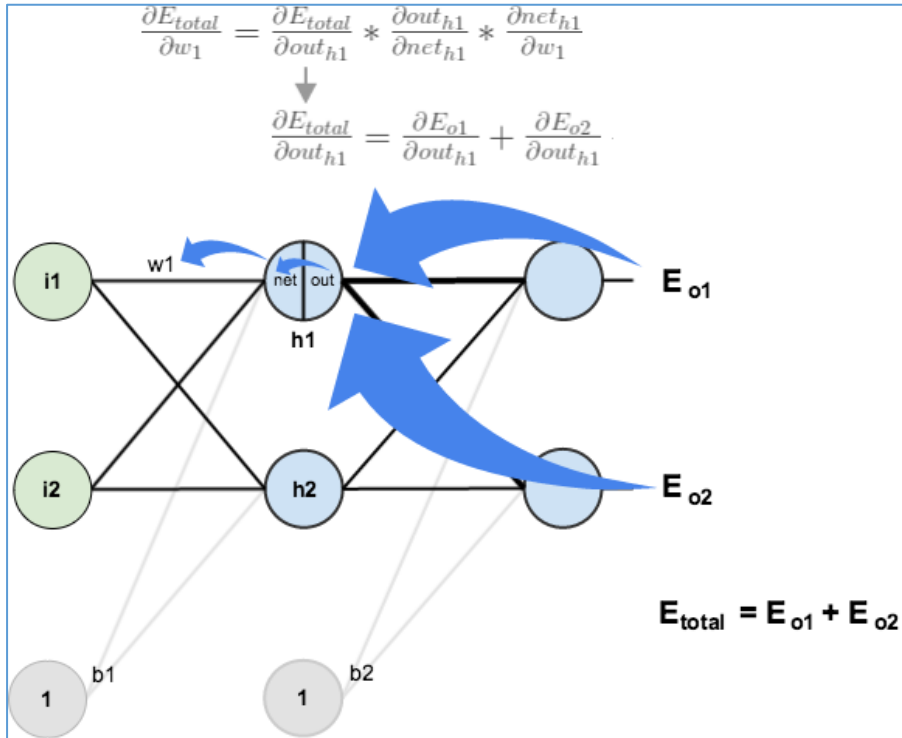
$$2. \quad \frac{\partial E_{o1}}{\partial net_{o1}} = \frac{\partial E_{o1}}{\partial out_{o1}} * \frac{\partial out_{o1}}{\partial net_{o1}} = 0.74136507 * 0.186815602 = 0.138498562$$

$$3. \quad \frac{\partial net_{o1}}{\partial out_{h1}} = w_5 = 0.40$$

$$4. \quad \frac{\partial E_{o1}}{\partial out_{h1}} = \frac{\partial E_{o1}}{\partial net_{o1}} * \frac{\partial net_{o1}}{\partial out_{h1}} = 0.138498562 * 0.40 = 0.055399425$$

$$5. \quad \frac{\partial E_{total}}{\partial out_{h1}} = \frac{\partial E_{o1}}{\partial out_{h1}} + \frac{\partial E_{o2}}{\partial out_{h1}} = 0.055399425 + -0.019049119 = 0.036350306$$

Step – 3: Backward Propagation to one hidden Layer



Calculation of $\frac{\partial out_{h1}}{\partial net_{h1}}$

- $$out_{h1} = \frac{1}{1 + e^{-net_{h1}}}$$

$$\frac{\partial out_{h1}}{\partial net_{h1}} = out_{h1}(1 - out_{h1}) = 0.59326999(1 - 0.59326999) = 0.241300709$$

Calculation of $\frac{\partial net_{h1}}{\partial w_1}$

$$net_{h1} = w_1 * i_1 + w_3 * i_2 + b_1 * 1$$

$$\frac{\partial net_{h1}}{\partial w_1} = i_1 = 0.05$$

Step – 3: Backward Propagation to one hidden Layer

Putting it all together:

$$\frac{\partial E_{total}}{\partial w_1} = \frac{\partial E_{total}}{\partial out_{h1}} * \frac{\partial out_{h1}}{\partial net_{h1}} * \frac{\partial net_{h1}}{\partial w_1}$$

$$\frac{\partial E_{total}}{\partial w_1} = 0.036350306 * 0.241300709 * 0.05 = 0.000438568$$

We can now update w_1 :

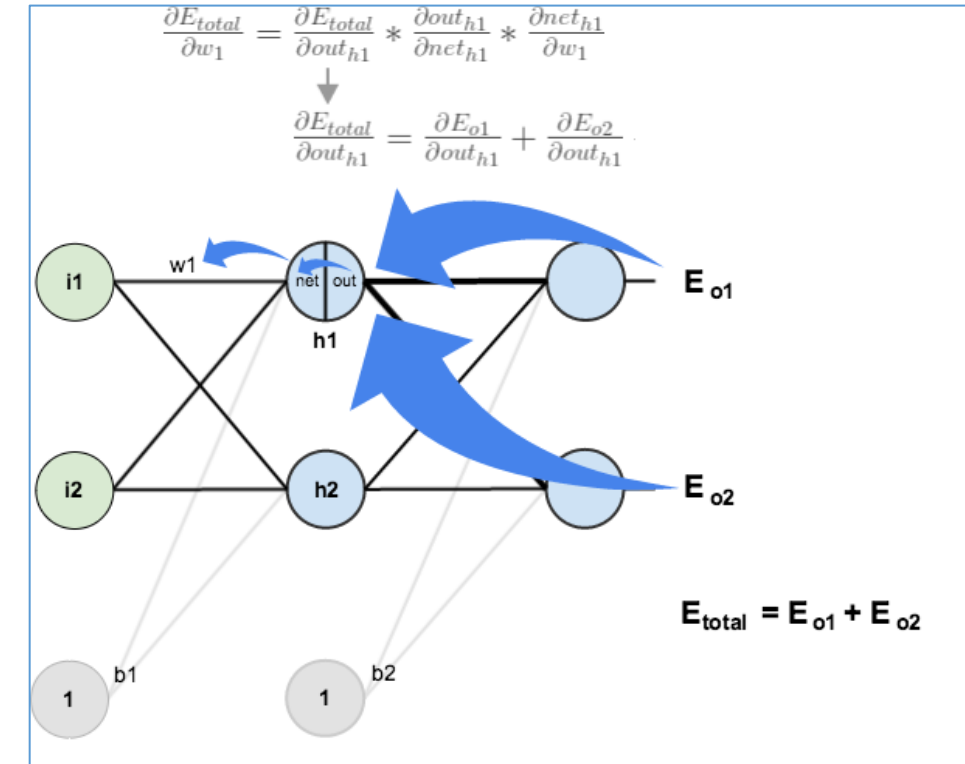
$$w_1^+ = w_1 - \eta * \frac{\partial E_{total}}{\partial w_1} = 0.15 - 0.5 * 0.000438568 = 0.149780716$$

Repeating this for w_2 , w_3 , and w_4

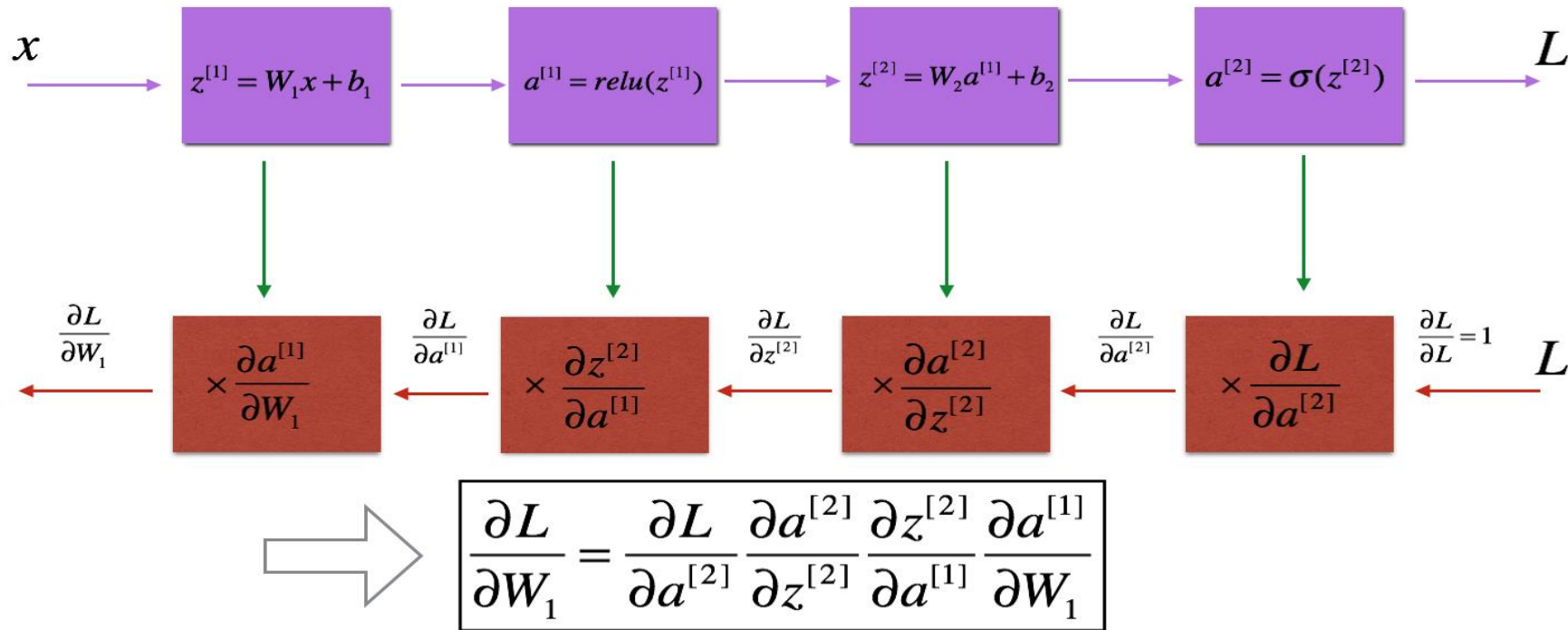
$$w_2^+ = 0.19956143$$

$$w_3^+ = 0.24975114$$

$$w_4^+ = 0.29950229$$



After this first round of backpropagation, the total error is now down to 0.291027924 (from 0.298371109). But after repeating this process 10,000 times, the error comes down to 0.0000351085 with the two outputs neurons generate 0.015912196 (vs 0.01 target) and 0.984065734 (vs 0.99 target).



We know that propagation is used to calculate the gradient of the loss function with respect to the parameters.

- <https://www.edureka.co/blog/backpropagation/>
- <https://www.youtube.com/watch?v=QJoa0JYaX1I>
- <https://www.analyticsvidhya.com/blog/2020/07/neural-networks-from-scratch-in-python-and-r/>



Q & A